

3주차_5장: CNN

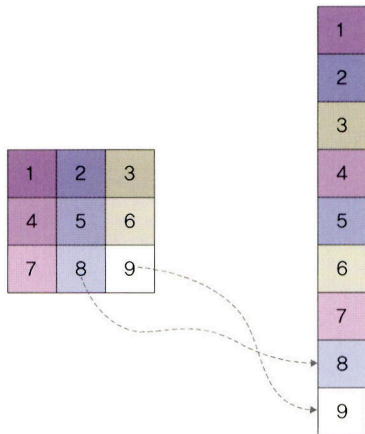
5.1 합성곱 신경망

- 딥러닝 역전파: 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드(출력층 → 은닉층 → 입력층)으로 전송됨
- 문제: 계산 과정은 많은 자원, 시간을 요구함
- 해결: 합성곱 신경망; 이미지의 국소적인 부분을 계산 → 시간과 자원을 절약해 분석할 수 있음

5.1.1 합성곱층의 필요성

- 3 X 3 흑백 이미지 처리 과정
 - 3 X 3 배열을 펼쳐(flattening) 각 픽셀에 가중치를 곱해 은닉층으로 전달
 - 이미지 펼쳐 분석하면 데이터의 공간적 구조를 무시하게 됨; 해결: 방지하기 위해 합성곱층 도입

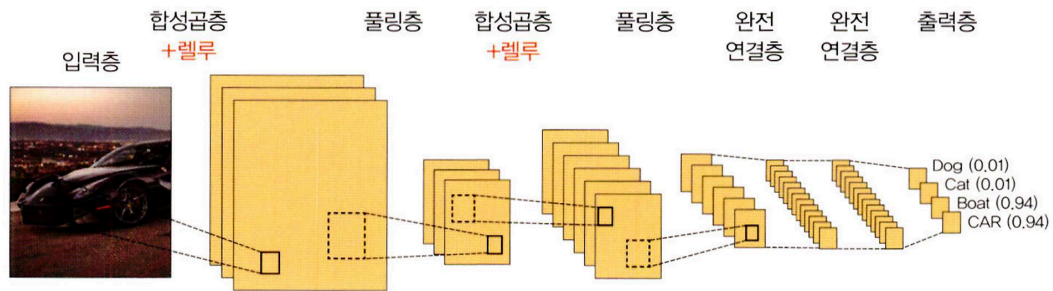
▼ 그림 5-1 합성곱층 원리



5.1.2 합성곱 신경망 구조

- 합성곱 신경망(Convolutional Neural Network, CNN, ConvNet)은 음성 인식이나 이미지/영상 인식에서 주로 사용하는 신경망
- 다차원 배열을 처리하도록 구성됨, 컬러 이미지 같은 다차원 배열 처리에 특화되어 있음
- 구성: 다섯 개의 계층
 - 입력층
 - 합성곱층
 - 풀링층
 - 완전연결층
 - 출력층

♥ 그림 5-2 합성곱 신경망 구조

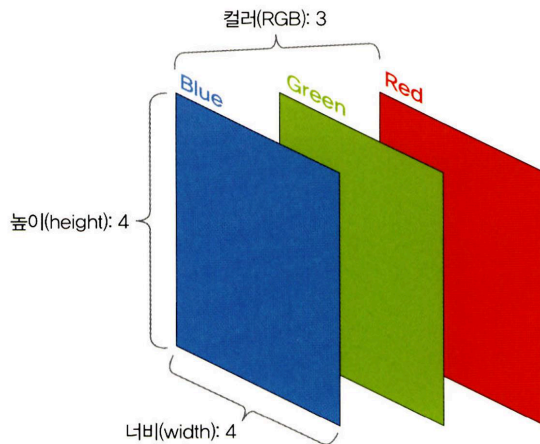


- 합성곱층과 풀링층을 거치며 입력 이미지의 주요 특성 벡터(feature vector)를 추출
- 추출된 주요 특성 벡터들은 완전 연결층을 거치며 1차원 벡터로 변환
- 출력층에서 활성화 함수 softmax 함수를 이용해 최종 결과 출력

입력층(input layer)

- 이미지: height, width, channel을 갖는 3차원 데이터
 - 이미지가 gray scale이면 1 값, 컬러(RGB)이면 3 값
 - 높이 4, 너비 4, 채널 RGB → 이미지 shape (4, 4, 3)

♥ 그림 5-3 채널



합성곱층(convolution layer)

- 입력 데이터에서 특성을 추출하는 역할 수행
- 그림 특성 추출: 입력 이미지 들어왔을 때 이미지에 대한 특성 감지를 위해 kernel이나 필터 사용
- 커널/필터는 특성 추출; 특성 결과물은 특성 맵(feature map)
- 커널 3x3, 5x5 크기 적용 일반적
- 스트라이드(stride)에 따라 순차적으로 이동
- 스트라이드가 1일 때 이동하는 과정
 1. 입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더함

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

▼ 그림 5-4 입력 이미지에 3×3 필터 적용



2. 필터가 1만큼 이동

▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



3. 필터가 1만큼 두 번째로 이동

▼ 그림 5-6 입력 이미지에 필터가 1만큼 두 번째 이동



4. 필터 1만큼 세 번째로 이동

▼ 그림 5-7 입력 이미지에 필터가 1만큼 세 번째 이동



5. ... 1만큼 마지막으로 이동

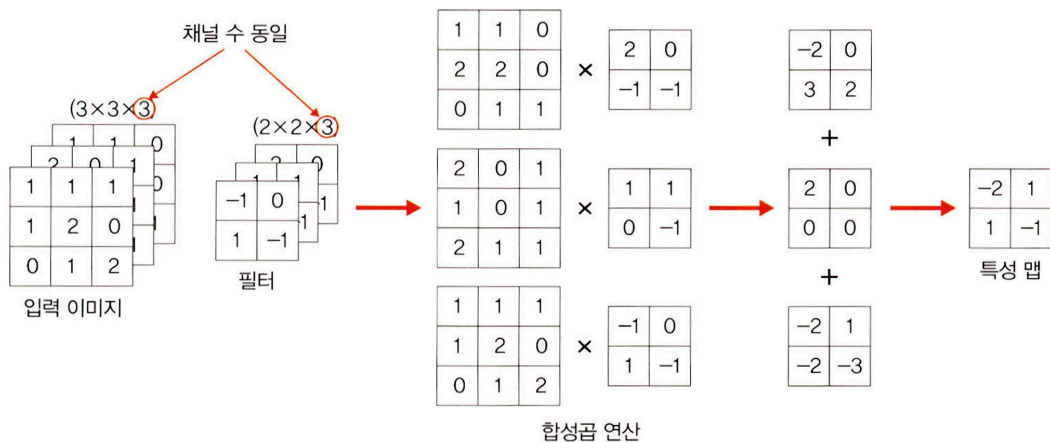
▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동



⇒ 이미지 크기 (6, 6, 1), 3x3 크기의 커널/필터가 스트라이드 1 간격으로 이동하며 합성곱 연산 수행하고 있음
원본 (6, 6, 1) 크기가 (4, 4, 1) 크기의 특성 맵으로 줄어들음

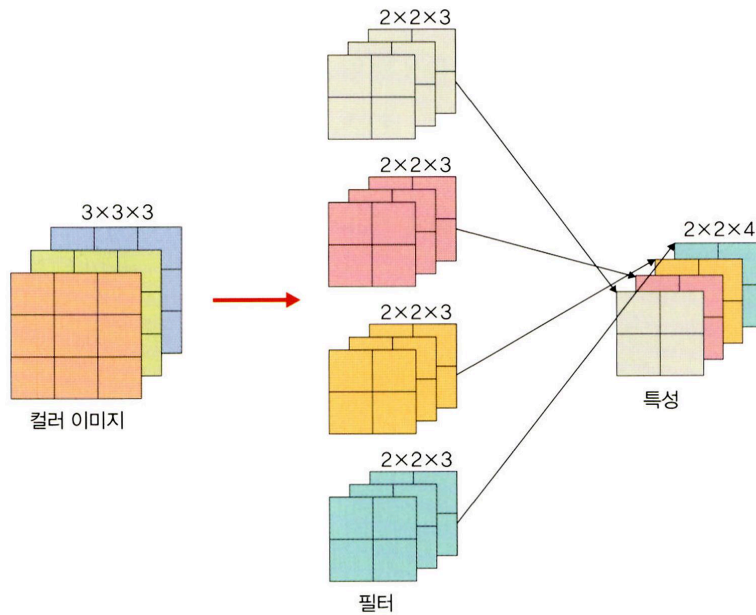
- 컬러 이미지의 합성곱
 - 필터 채널링 3
 - 필터 채널은 3이나 필터 개수는 하나
 - RGB 각각 서로 다른 가중치로 합성곱 적용 후 더함

▼ 그림 5-10 컬러 이미지 합성곱



- 필터가 두 개 이상인 합성곱: 필터 각각은 특성 추출 결과의 채널이 됨

▼ 그림 5-11 필터가 2 이상인 합성곱



- 입력 데이터: W_1 (가로) x H_1 (세로) x D_1 (채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/s + 1$
 - $D_2 = K$

풀링층(pooling layer)

- 특성 맵의 차원을 다운 샘플링 하여 연산량 감소시키며, 주요한 특성 벡터를 추출

▼ 그림 5-12 다운 샘플링



- 최대 풀링(max pooling): 대상 영역에서 최댓값 추출
- 평균 풀링(average pooling): 대상 영역에서 평균 반환

- 대부분 합성곱 신경망에서 최대 풀링 사용
 - 평균 풀링은 각 커널 값을 평균화 → 중요한 가중치를 갖는 값의 특성이 희미해질 수 있음
1. 첫 번째 최대 풀링 과정

3, -1, -3, 1 값 중에서 최댓값(3)을 선택합니다.

▼ 그림 5-13 첫 번째 최대 풀링 과정

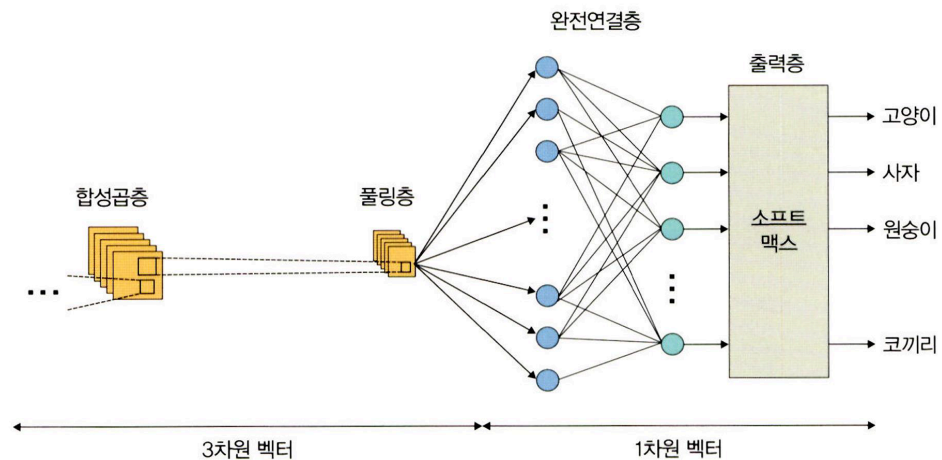


- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼 파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

완전 연결층

- 3차원 벡터에서 1차원 벡터로 펼쳐짐(flatten)

▼ 그림 5-18 완전연결층



출력층

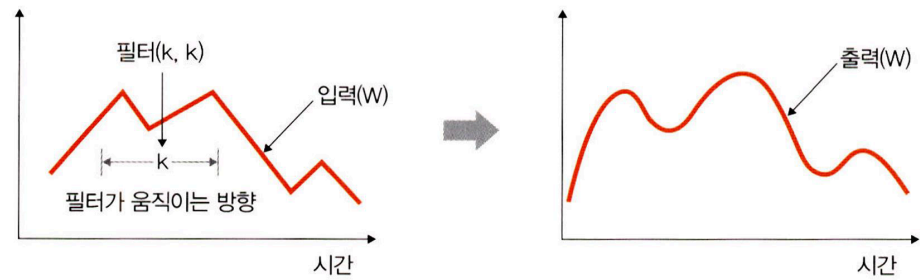
- softmax 활성화 함수 사용 → 0~1 사이의 값으로 출력
- 이미지가 각 레이블에 속할 확률 값 출력; 가장 높은 확률 값 선정

5.1.3 1D, 2D, 3D 합성곱

1D 합성곱

- 필터가 시간을 축으로 좌우로만 입력
- 입력(W)과 필터(k)에 대한 출력은 $W-k+1$

▼ 그림 5-19 1D 합성곱

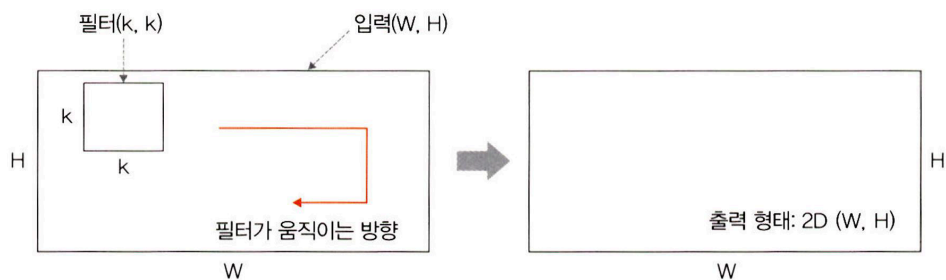


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

2D 합성곱

- 두 개의 방향으로 움직임
- 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H); 출력 형태는 2D 행렬

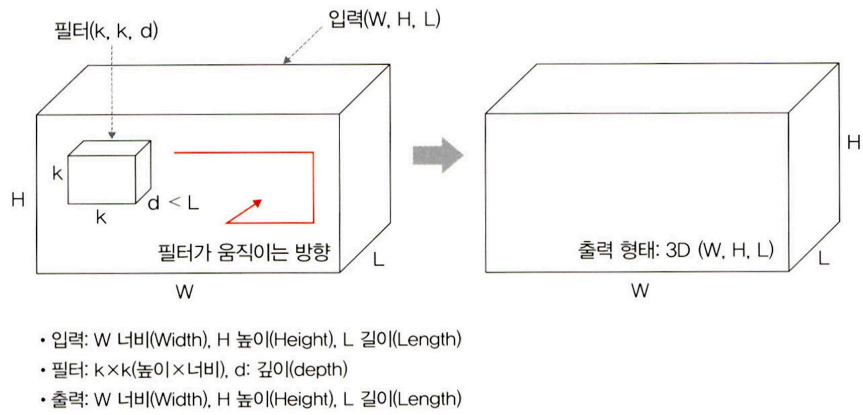
▼ 그림 5-20 2D 합성곱



- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

3D 합성곱

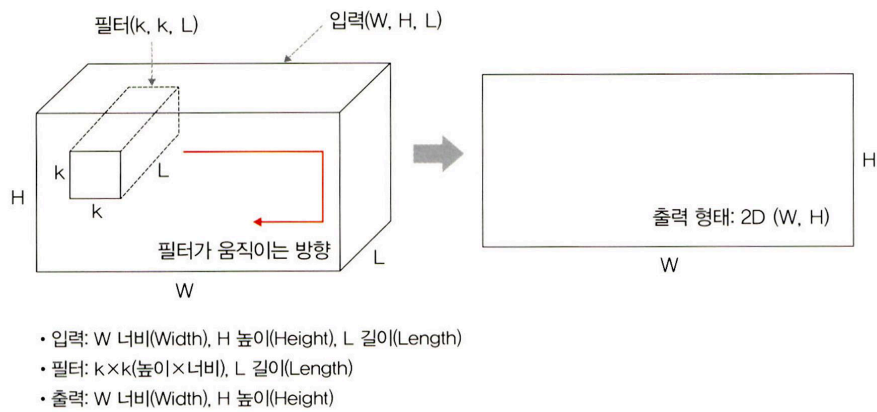
- 필터가 세 개의 방향으로 움직임
- 입력(W, H, L)에 필터(k, k, d) 적용 $\rightarrow$ 출력 (W, H, L) 형태의 3D 합성곱
- $d < L$ 유지해야함



3D 입력을 갖는 2D 합성곱

- 필터에 대한 길이(L)와 입력 채널의 길이(L)의 길이가 같아 생김
- 입력 (W, H, L)에 필터(k, k, L) 적용하면 출력은 (W, H)
- LeNet-5, VGG

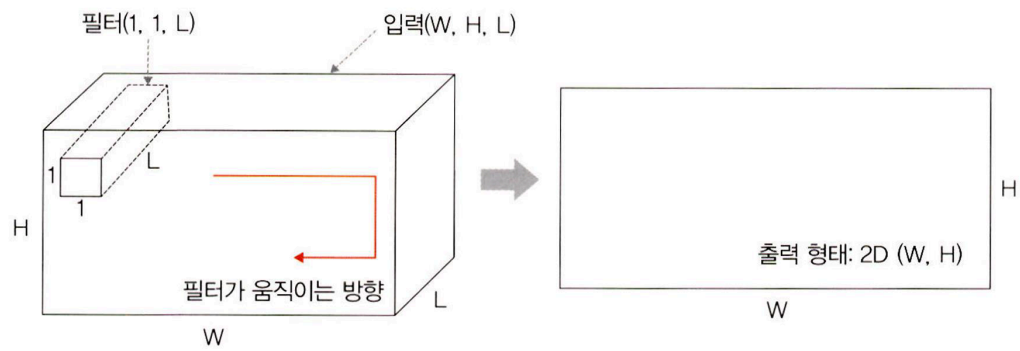
▼ 그림 5-22 3D 입력을 갖는 2D 합성곱



1×1 합성곱

- 3D 형태로 입력
- 입력 (W, H, L)에 필터($1, 1, L$) 적용 → 출력(W, H)
- GoogLeNet

♥ 그림 5-23 1×1 합성곱



- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: 1×1(높이×너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

5.2 합성곱 신경망 맛보기(199)

- fashion_mnist 데이터셋을 이용한 합성곱 신경망 구현

fashion_mnist 데이터셋

운동화, 셔츠, 샌들과 같은 작은 이미지 모음

10가지로 분류될 수 있는 28 x 28 이미지 7만 개

훈련 데이터(train_images)는 0에서 255 사이의 값을 갖는 28 x 28 크기의 NumPy 배열

레이블(정답) 데이터(train_labels)는 0에서 9까지 정수 값 가짐

- 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
```

- CPU, GPU 장치 확인

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

GPU 사용

하나의 GPU 사용하는 경우

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = Net()
model.to(device)
```

다수의 GPU 사용하는 경우 nn.DataParallel 이용

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = Net()
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model)
model.to(device)
```

batch size가 알아서 GPU로 분배됨

- fashion_dataset 내려받기

```
train_dataset = torchvision.datasets.FashionMNIST(r"C:\Users\eunsu\학교\2026-2\euron\DL_주차별_활동자료\3주차_5장\colab_5장.ipynb", download=True, transform=transforms.Compose([transforms.ToTensor()]))
test_dataset = torchvision.datasets.FashionMNIST(r"C:\Users\eunsu\학교\2026-2\euron\DL_주차별_활동자료\3주차_5장\colab_5장.ipynb", download=True, train=False, transform=transforms.Compose([transforms.ToTensor()]))
```

- torchvision.datasets
 - torch.utils.data.Dataset의 하위 클래스
 - `train_dataset = torchvision.datasets.FashionMNIST(r"C:\Users\eunsu\학교\2026-2\euron\DL_주차별_활동자료\3주차_5장\colab_5장.ipynb", download=True, transform=transforms.Compose([transforms.ToTensor()]))`
 - FashionMNIST 내려받을 위치 지정
 - download: download True로 변경 → 첫 번째 파라미터 위치에 해당 데이터셋 있는지 확인 후 내려받음
 - transform: 이미지 텐서(0~1)로 변경
- fashion_mnist 데이터를 DataLoader에 전달

```
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size = 100)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size = 100)
```

- torch.utils.data.DataLoader() 사용해 원하는 크기의 배치 단위로 데이터 불러옴 + shuffle
 - `torch.utils.data.DataLoader(train_dataset, batch_size=100)`
 - train_dataset: 데이터를 불러올 데이터셋 지정
 - batch_size: 데이터를 배치로 묶어줌
- 분류에 사용될 클래스 정의

```
labels_map = {0: 'T-Shirt', 1: 'Trouser', 2: 'Pullover', 3: 'Dress', 4: 'Coat', 5: 'Sand-  
dal', 6: 'Shirt', 7: 'Sneaker', 8: 'Bag', 9: 'Ankle Boot'}

fig = plt.figure(figsize=(8,8));
columns = 4;
rows = 5;
for i in range(1, columns*rows+1):
    img_xy = np.random.randint(len(train_dataset));
    img = train_dataset[img_xy][0][0, :, :]
    fig.add_subplot(rows, columns, i)
    plt.title(labels_map[train_dataset[img_xy][1]])
    plt.axis('off')
```

```
plt.imshow(img, cmap = 'gray')
plt.show()
```

np.random: data 무작위 생성 시 사용

```
import numpy as np
np.random.randint(10) # 0~ 9 임의의 숫자 출력

np.random.randint(1, 10) # 1~9 임의의 숫자 출력

np.random.rand(8) # 0~1 사이 정규표준분포 난수를 행렬로 (1 x 8) 출력

np.random.rand(4, 2)

np.random.randn(8) #평균이 0, 표준편차 1인 가우시안 정규분포 난수 행렬 (1x8) 출력
```

- 합성곱 신경망이 아닌 심층 신경망 학습

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features = 256)
        self.drop= nn.Dropout(0.25)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data):
        out = input_data.view(-1, 784)
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

- def __init__(self): 클래스 모델은 항상 torch.nn.Module을 상속 받음
 - __init__(): 객체가 갖는 속성 값을 초기화하는 역할; 객체가 생성될 때 자동으로 호출됨
 - super(FashionDNN, self).__init__(): FashionDNN이라는 super 클래스를 상속 받았다는 의미

객체 지향 프로그래밍(object oriented programming)

- 프로그램에서 필요한 데이터를 추상화하여 속성이나 행동, 동작, 특징 등을 객체로 만들고 객체들이 유기적으로 동작하도록 하는 프로그래밍 방법
- 객체(object): 메모리를 할당받아 프로그램에서 사용되는 모든 데이터
- 객체명 = 클래스명()

클래스와 함수

- function: 하나의 특정 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드
- 클래스는 함수 + 변수 한꺼번에 묶어서 관리하고 재사용할 수 있게 해줌

- 함수

```
def add(num1, num2):
    result = num1 + num2
    return result

print(add(1, 2))
print(add(2, 3))
```

- 클래스

```
class Calc:
    def __init__(self):
        self.result = 0

    def add(self, num1, num2):
        self.result = num1 + num2
        return self.result

obj1 = Calc()
obj2 = Calc()

print(obj1.add(1, 2))
print(obj1.add(2, 3))
print('-----')
print(obj2.add(2, 2))
print(obj2.add(2, 3))
```

- nn은 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지
 - Linear: 단순 선형 회귀 모델 만들 때 사용
 - nn.Linear(in_features, out_features)
 - in_features: 입력의 크기
 - out_features: 출력의 크기
- torch.nn.Dropout(p): p만큼의 비율로 텐서 값이 0이 됨
 - 0이 되지 않는 기존 값들은 $(1/(1-p))$ 만큼 곱해져 커짐
- forward(): 순전파(forward propagation) 학습을 진행시킴
 - 모델이 학습 데이터를 입력받아 순전파 연산을 진행하는 함수
 - 객체를 데이터와 함께 호출하면 자동으로 실행됨
 - 순전파 연산: $H(x)$ 식에 입력 x로부터 예측된 y 얻기
- view: 넘파이의 reshape과 같은 역할
 - 텐서의 크기(shape)을 변경해주는 역할
 - input_data.view(-1, 784)는 input_data를 (7,784)의 크기로 변경함
- 활성화 함수 지정 방법
 - F.relu(): forward 함수에서 정의
 - nn.ReLU(): __init__() 함수에서 정의

nn.functional.xx()(or F.xx())와 nn.xx()의 차이

- nn 사용 코드

```
import torch
import torch.nn as nn

inputs = torch.randn(64, 3, 244, 244)
conv = nn.Conv2d(in_channels = 3, out_channels=64, kernel_size=3, padding=1)
outputs = conv(inputs)
layer = nn.Conv2d(1, 1, 3)
```

- nn.functional 사용 코드

```
import torch.nn.functional as F

inputs = torch.randn(64, 3, 244, 244)
weight = torch.randn(64, 3, 3, 3)
bias = torch.randn(64)
outputs = F.conv2d(inputs, weight, bias, padding=1)
```

- nn.Conv2d에서 input_channel과 output_channel을 사용해 연산
- functional.conv2d는 입력(input)과 가중치(weight) 자체를 직접 넣어줌
 - "자체로 직접 넣어준다": 가중치를 전달할 때마다 가중치의 값을 새로 정의해야함

▼ 표 5-1 nn.xx와 nn.functional.xx의 사용 방법 비교

구분	nn.xx	nn.functional.xx
형태	nn.Conv2d: 클래스 nn.Module 클래스를 상속받아 사용	nn.functional.conv2d: 함수 def function (input)으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	nn.Sequential 내에 위치	nn.Sequential에 위치할 수 없음
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

- 모델 학습 전 손실함수, 학습률, 옵티마이저에 대해 정의

```
learning_rate = 0.001
model = FasionDNN()
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate)
print(model)
```

- optimizer 경사 하강법 Adam 사용
- 학습률 lr 0.001 사용
- 심층 신경망을 이용한 모델 학습

```

num_epochs = 5
count = 0

loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        label = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    # 50회 배치마다 검증 (train_loader 루프 안으로 이동)
    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

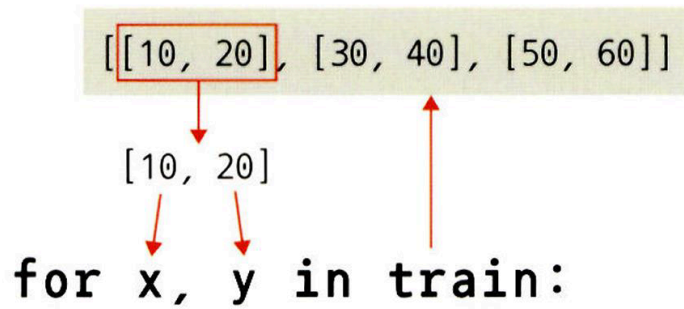
        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    # 500회 배치마다 출력 (train_loader 루프 안으로 이동)
    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss.data,
accuracy))

```

- 배열이나 행렬과 같은 리스트 사용하는 방법
 - loss_list[]와 같이 비어 있는 배열이나 행렬 만들
 - append 메서드 이용해 데이터 추가
- for images, labels in train_loader:
 - for 구문 사용해 레코드 하나씩 가져옴

♥ 그림 5-26 for in 구문



- `images, labels = images.to(device), labels.to(device)`
 - 모델이 데이터를 처리하기 위해 모델과 데이터가 동일한 장치(CPU 또는 GPU)에 있어야함
- `train = Variable(images.view(100, 1, 28, 28))`
 - Autograd: 자동 미분을 수행하는 파이토치 핵심 패키지
 - 자동 미분에 대한 값을 저장하기 위해 `tape` 사용
 - 순전파(forward) 단계에서 테이프는 수행하는 모든 연산 저장
 - 역전파(backward) 단계에서 저장된 값 꺼내 사용
 - `Variable`을 이용해 역전파를 위한 미분 값 자동으로 계산
 - 따라서 `torch.autograd` 패키지 안 `Variable`을 이용해야함
- `accuracy = correct * 100 / total`
 - 분류 문제에 대한 정확도
 - `error rate = (1-(correct predictions / total predictions)) * 100`
 - 분류 문제에서의 정확도 주의 사항
 - 정확도의 값이 모든 클래스가 동등하게 고려된 것인지 유의해야함
 - 데이터 특징 유의(데이터의 90%가 하나의 클래스에 속하는 경우 데이터가 하나의 클래스에 속한다고 예측해도 예측 결과가 90%가 나옴)
- `loss_list.append(loss.data)`
- 합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size = 3, padding =1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc1 = nn.Linear(in_features=64*6*6, out_features = 600)
        self.drop = nn.Dropout2d(0.25)
```

```

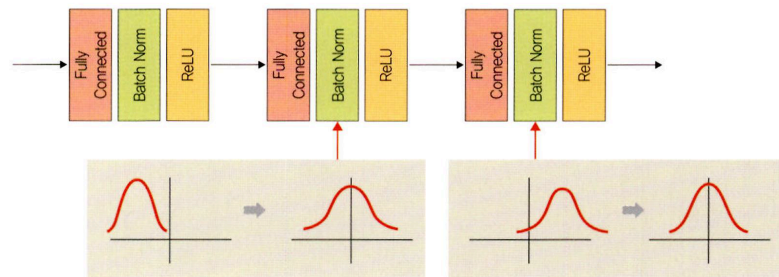
self.fc2 = nn.Linear(in_features=600, out_features=120)
self.fc3 = nn.Linear(in_features=120, out_features=10)

def forward(self, x):
    out = self.layer1(x)
    out = self.layer2(out)
    out = out.view(out.size(0), -1)
    out = self.fc1(out)
    out = self.drop(out)
    out = self.fc2(out)
    out = self.fc3(out)
    return out

```

- self.layer1 = nn.Sequential
 - nn.Sequential 사용 → `__init__()`에 사용할 네트워크 모델 정의
 - forward() 함수에서 구현될 순전파를 layer 형태로 가독성 있게 코드 작성할 수 있음
 - layer 차례대로 쌓을 수 있도록 $Wx+b$ 와 같은 수식과 활성화 함수 연결해줌
- nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
 - 합성곱: 커널(또는 필터)이라는 $n \times m$ 크기의 행렬이 높이(height) X 너비(width) 크기의 이미지를 처음부터 끝까지 훑으며 각 원소끼리 곱한 후 모두 더한 값 출력
 - in_channels: 입력 채널의 수; 흑백 이미지는 1, RGB 값은 3
 - out_channels: 출력 채널 수
 - kernel_size: 커널 크기, 필터. 이미지 특징을 찾아내기 위한 고용 파라미터
- nn.BatchNorm2d(32)
 - 배치별 평균과 분산을 이용해 정규화

♥ 그림 5-28 BatchNorm2d



- nn.MaxPool2d(kernel_size=2, stride=2)
 - 이미지 축소
 - 풀링 계층: 합성곱층의 출력 데이터를 입력으로 받아 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용
 - 풀링 계층 처리 방법 최대 풀링(max pooling), 평균 풀링(average pooling), 최소 풀링(min pooling)
 - kernel_size: $n \times m$ 행렬로 구성된 가중치
 - stride: 입력 데이터에 커널(필터) 적용할 때 이동 간격; 스트라이드 값 커질수록 출력 크기 작아짐
- self.fc1 = nn.Linear(in_features = 64*6*6, out_features=600)
 - 클래스 분류 위해 이미지 형태의 데이터 배열 형태로 변환해야함
 - Conv2d에서 사용하는 하이퍼파라미터(패딩, 스트라이드) 값에 따라 출력 크기(output size)가 달라짐

- in_features: 입력 데이터의 크기; Conv2d, MaxPool2d 출력 결과를 완전 연결층으로 보내기 위해 1차원으로 변경해줘야함

Conv2d 계층에서 출력 크기 구하기

출력 크기 = $(W - F + 2P) / S + 1$

- W: 입력 데이터 크기(input_volume_size)
- F: 커널 크기(kernel_size)
- P: 패딩 크기(padding_size)
- S: 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하기

출력 크기 = IF / F

- IF: 입력 필터의 크기(input_filter_size; Conv2d의 출력 크기)
- F: 커널 크기(kernel_size)

- out = out.view(out.size(0), -1)
 - 합성곱층에서 완전 연결층으로 변경됨 → 데이터의 형태를 1차원으로 바꿔줌
 - out.size(0) : 100 의미
 - out.view(out.size(0), -1)에서 -1 열 수 알지 못할 때 사용
- 합성곱 네트워크를 위한 파라미터 정의

```
learning_rate = 0.001
model = FashionCNN()
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr= learning_rate)
print(model)
```

- 모델 학습 및 성능 평가

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
```

```

optimizer.step()
count += 1

if not (count % 50):
    total = 0
    correct = 0
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        labels_list.append(labels)
        test = Variable(images.view(100, 1, 28, 28))
        outputs = model(test)
        predictions = torch.max(outputs, 1)[1].to(device)
        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

if not (count % 500):
    print("Iteration: {}, Loss: {}, Accuracy: {}".format(count, loss_data, accuracy))

```

5.3 전이 학습

5.4 설명 가능한 CNN

5.5 그래프 합성곱 네트워크